

C

En introduktion.

Johan Havner
Niklas Mårdby
Daniel Winckler

Lite kuriosa om C.

- *The C Programming Language*, 1978
- ANSI C, 1983.
- UNIX är skrivet i C.
- Med C kan man göra i princip allt man vill, om man kan. Det är ett stort om.

Ett litet program.

include <stdio.h>

inkludera bibliotek

<i>main()</i>	<i>/* Funktionen main */</i>	definiera en funktion som heter main
{		satserna i main står inom {}
<i>printf("Hello, human!\n");</i>		printf skriver ut strängar
}		} avslutar funktionen main

Text mellan */** och **/* ignoreras av
kompilatorn, används till att
kommentera koden.

Även *//* kan användas, resten av
raden ignoreras.

Vad gör man sen då?

I NC och MC så kompilerar man antingen med *cc* eller *gcc*.

Om vi har koden från föregående OH sparad som en vanlig textfil med namnet *test.c* så skriver vi:

gcc test.c -o test

Flaggan *-o* namnger outputen, den exekverbara filen vi får heter *test*.

Variabler och aritmetik.

```
#include <stdio.h>
main()
{
    int a, b;
    a = 0;
    b = 5;
    while(a <= b){
        printf("%d\t", a, b);
        a = a + 1;
    }
    printf("\n");
}
```

Kodsnutten visar lite olika saker:

- Output: 0 1 2 3 4 5
- Variabler:
int a, b;
- Tilldelning av värden:
a = 0;
b = 5;
- While-sats:
while(condition){
 ...
}
- Printf, mer om den på nästa OH.

Logiska operatorer.

==	lika med
!=	inte lika med
<	mindre än
<=	mindre än eller lika med
>	större än
>=	större än eller lika med
&&	och
	eller

Printf-funktionen.

<i>%d</i>	integer (decimal)
<i>%xd</i>	integer, at least x characters wide
<i>%f</i>	float
<i>%xf</i>	float, at least x characters wide
<i>%.xf</i>	float with x decimals
<i>%x.yf</i>	float, at least x characters, y decimals
<i>printf("%d\n", 300);</i>	300
<i>\n, \t, \b</i>	newline, tab, backspace

For-satsen.

```
#include <stdio.h>
main()
{
    int a;
    for(a=0; a<5; a=a+1)
    {
        printf("%d\t", a);
    }
    printf("\n");
}
```

Output:

0 1 2 3 4

For-satsen forts.

Formen för en for-loop:

```
for(int start; loop condition; loop expr)  
{  
    statements  
}
```

Till skillnad från i Java så får man inte initiera integern inuti for-satsen.

Java: *for(int a = 0; a<5; a=a+1)*

C: *int a;*
 for(a = 0; a<5; a=a+1)

Symboliska konstanter.

#define namn ersättningstext

Ex: #define MAXVAL 8

- Exemplet ersätter alla förekomster av MAXVAL i koden med 8.
- Symboliska konstanter gör att man med säkerhet vet att alla förekomster av MAXVAL har ersatts med 8.

If-satsen.

```
a = 2;  
if(a == 1){  
    printf("En etta.");  
}  
else if(a == 2){  
    printf("En tvåa.");  
}  
else{  
    printf("Varken en etta eller en tvåa.");  
}
```

If-satsen forts.

Formen för en if-sats:

```
if(condition1){  
    statements1  
}  
  
else if(condition2){  
    statements2  
}  
  
else if(condition3){  
    statements3  
}  
  
else{  
    statements4  
}
```

Arrays.

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    char a[4];
```

```
    int b;
```

```
    for(b=0; b<4; b++)
```

```
        a[b] = 'x';
```

```
    for(b=0; b<4; b++)
```

```
        printf("%c ", a[b]);
```

```
    printf("\n");
```

```
}
```

Output:

x x x x

Arrays forts.

- Indexerade 0 till n-1, där n är längden.
- Initieras med: *type name[number of elements];*
Exempel: *int digits[10];*
- I Java är String en typ men i C är en sträng en array med *char*, *char[]*, som avslutas med tecknet '\0'. (Vid filinläsning ska man använda sig av *int[]* istället då EOF inte får plats i en *char*.)

Funktioner.

```
return-type function-name(parameter declarations, if any)  
{  
    declarations  
    statements  
}
```

Exempel:

```
int add2(int n)  
{  
    int m;  
    m = n+2;  
    return m;  
}
```

Funktioner forts.

- return-type: vad funktionen ska returnera, void om inget
- function-name: vad funktionen heter
- parameter declarations: funktionens argument
- declarations: variabler som funktionen använder
- statements: saker som funktionen gör
- Alla funktioner utom *main()* måste deklareras i början av filen. Fördel med detta: överblick.
Den raden kallas för *function prototype*.

Funktioner forts.

```
#include <stdio.h>
```

```
int add2(int);
```

← *function prototype*

```
main(){
```

```
    int m;
```

```
    m = 2;
```

```
    printf("%d\n", add2(m));
```

```
}
```

```
int add2(int add2n){
```

```
    return add2n+2;
```

```
}
```

Man behöver inte namnge parametrarna,
bara deklarerera.

Argument - call by value.

```
main(){
```

```
int n1, n2, tot;
```

```
n1 = 5;
```

```
n2 = 3;
```

```
tot = incadd(n1, n2);
```

```
printf("%d+%d=%d\n", n1, n2, tot);
```

```
}
```

```
int incadd(int x, int y){
```

```
    x = x+1;
```

```
    y = y+1;
```

```
    return x+y;
```

```
}
```

(Inget komplett program.)

Output:

5+3=10

5 plus 3 blir 8, inte 10.

När *incadd* anropas så kopieras n1 och n2 i x och y. Detta leder till att n1 och n2 inte ändras.

Argument - call by value forts.

- En funktion skapar de variabler den behöver när den körs. Eventuella parametrar kopieras in.
- Icke så med arrays. Om en array ges som argument till en funktion så får funktionen full tillgång till arrayn.

Det var alltså allt. Nästan...

Det finns lite, lite till...

External variables. Det är globala variabler.
Man kan använda dem, det står i
föreläsningsanteckningarna hur man gör. Detta bör
dock undvikas då det otroligt lätt skapar oreda
utan like.

Slutord.

- DET BLIR ALDRIG RÄTT!
- Aldrig. Alls.
- Se ovan.
- Nej, inte. Inte. Nej.

C är kraftfullt. Det är sant. Dessvärre även otroligt oförlåtligt vad gäller misstag. Kompilatorn hittar långt från alla fel - tungan i rätt mun från början med andra ord.